

AI Voice Intelligence Platform MVP Documentation

1. Executive Summary

The AI Voice Intelligence Platform is an AI-powered backend application developed to analyze recorded conversations and automatically generate structured business insights. The platform is capable of processing sales calls, customer interviews, support conversations, and research discussions.

The primary objective of the project was to build a complete end-to-end pipeline that transforms raw audio recordings into actionable information through automated speech transcription, speaker identification, conversation summarization, insight extraction, and transcript-based question answering.

The system accepts audio files uploaded by users and processes them through multiple AI-powered stages. These stages include audio preprocessing, speech-to-text conversion, speaker diarization, summary generation, insight extraction, embedding creation, semantic retrieval, and persistent storage.

The completed MVP successfully demonstrates the feasibility of building a scalable voice intelligence solution using a combination of open-source AI models and cloud-based AI services. The project establishes a strong foundation for future enhancements such as real-time processing, advanced analytics, authentication, and enterprise integrations.

2. Project Objective

Organizations regularly conduct sales meetings, customer interviews, support calls, and research discussions. These conversations contain valuable business information; however, manually reviewing recordings is time-consuming and difficult to scale.

The purpose of this project was to develop an AI-powered platform capable of automatically processing recorded conversations and extracting meaningful information without requiring manual analysis.

The primary objectives of the project were:

- Accept audio recordings through an API interface
- Convert speech into accurate text transcripts
- Identify multiple speakers participating in a conversation
- Generate concise and detailed conversation summaries
- Extract business insights such as pain points, objections, buying intent, and action items
- Enable users to ask questions about the transcript using semantic search
- Store all processed data and metadata for future retrieval

The overall goal was to create a functional MVP demonstrating the practical application of modern AI technologies in voice intelligence and conversation analytics.

3. Analysis & Technology Evaluation

Before implementation, several technologies and approaches were evaluated to identify the most suitable components for the MVP.

3.1 Speech-to-Text Evaluation

Faster-Whisper

Faster-Whisper was evaluated as the primary local transcription engine.

Advantages:

- Completely free to use
- Runs locally without cloud dependency
- Good transcription accuracy
- Supports multiple languages
- Easy integration with Python

Challenges:

- CPU-intensive for large recordings
- Processing time increases significantly for lengthy audio files
- Accuracy may be slightly lower than premium cloud services

AssemblyAI Universal-3-Pro

AssemblyAI was evaluated as a cloud-based transcription alternative.

Advantages:

- High transcription accuracy
- Built-in speaker diarization
- Faster processing

Challenges:

- Requires API usage
- Ongoing operational cost

Decision:

- The final implementation supports both Faster-Whisper and AssemblyAI, allowing flexibility between cost-free local processing and higher-accuracy cloud processing.

3.2 Speaker Diarization Evaluation

Speaker diarization is responsible for identifying different speakers within a conversation.

Pyannote Audio

Model Used:

- pyannote/speaker-diarization-3.1

Advantages:

- State-of-the-art open-source solution
- High speaker separation accuracy
- Fully local deployment

Challenges:

- Requires additional configuration
- Computationally intensive
- Longer processing time

AssemblyAI Diarization

Advantages:

- Built-in integration with transcription
- Minimal setup requirements
- High accuracy

Challenges:

- API dependency

Decision:

- The system supports both Pyannote and AssemblyAI diarization while also providing a fallback heuristic method when advanced diarization services are unavailable.

3.3 Large Language Model Evaluation

The platform required a language model capable of:

- Generating summaries
- Extracting business insights
- Supporting transcript-based question answering

After evaluating multiple options, Groq's Llama 3.1 8B Instant model was selected.

Reasons for Selection:

- Extremely low latency
- Fast response generation
- Reliable structured outputs
- Cost-effective for MVP development
- Suitable for real-time business applications

3.4 Vector Search Evaluation

To enable semantic transcript search, a vector retrieval system was required.

Sentence Transformers

Model Used:

- all-MiniLM-L6-v2

Reasons:

- Lightweight model
- Fast embedding generation
- Strong semantic understanding

FAISS

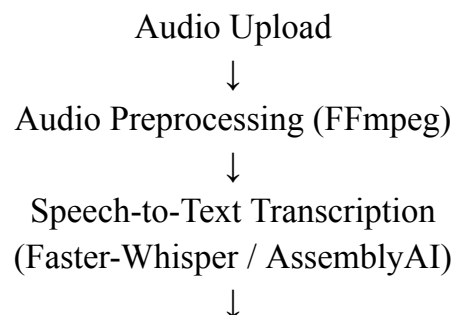
Reasons:

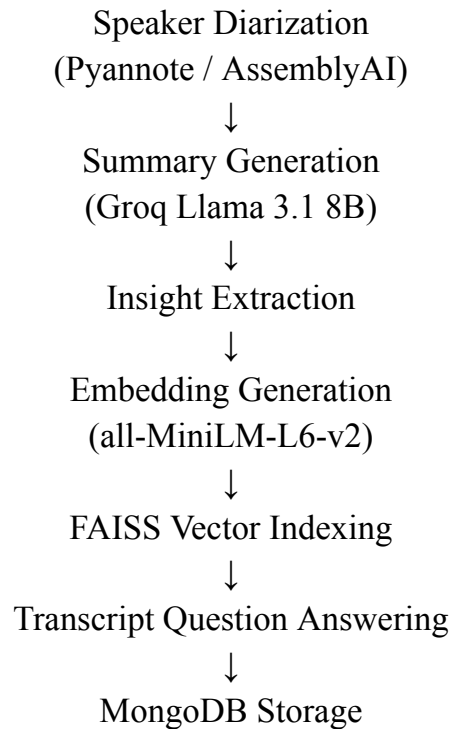
- Extremely fast vector similarity search
- Local deployment
- No external vector database requirement
- Efficient memory utilization

The combination of Sentence Transformers and FAISS provided an effective retrieval mechanism for transcript question answering.

4. Solution Architecture

System Workflow





Data Flow

Step 1:

User uploads an audio recording through the API.

Step 2:

The audio is normalized and converted using FFmpeg.

Step 3:

Speech transcription converts spoken language into text.

Step 4:

Speaker diarization assigns speaker labels to transcript segments.

Step 5:

The transcript is sent to the LLM for summary generation.

Step 6:

Business insights are extracted from the conversation.

Step 7:

Transcript embeddings are generated.

Step 8:

FAISS creates a searchable vector index.

Step 9:

All outputs are stored in MongoDB.

Step 10:

Users can perform transcript-based question answering.

5. Technology Stack

Backend Development

- Python 3.10+
- FastAPI
- Uvicorn
- Python Multipart

Audio Processing

- FFmpeg
- Torchaudio

Speech Recognition

- Faster-Whisper
- AssemblyAI Universal-3-Pro

Speaker Diarization

- Pyannote Audio
- AssemblyAI

Artificial Intelligence

- Groq Llama 3.1 8B Instant
- Sentence Transformers

Semantic Retrieval

- FAISS

Database

- MongoDB

6. Implementation Overview

The platform was developed as a modular backend system consisting of several independent components.

Audio Upload Module

Responsibilities:

- File upload handling
- Input validation
- Metadata creation
- Processing initiation

Audio Preprocessing Module

Responsibilities:

- Format conversion
- Sample rate normalization
- Audio cleanup

Tools:

- FFmpeg

Transcription Module

Responsibilities:

- Speech recognition
- Timestamp generation

Supported Engines:

- Faster-Whisper
- AssemblyAI

Speaker Diarization Module

Responsibilities:

- Speaker identification
- Segment assignment

Supported Methods:

- Pyannote
- AssemblyAI
- Heuristic fallback

AI Analysis Module

Responsibilities:

- Summary generation
- Key point extraction
- Pain point identification
- Objection detection
- Buying intent analysis
- Action item extraction

Semantic Search Module

Responsibilities:

- Transcript chunking
- Embedding generation
- Similarity search
- Question answering

Storage Module

Responsibilities:

- Transcript storage
- Analysis storage
- Metadata persistence

Database:

- MongoDB

7. API Endpoints

Method	Endpoint	Description
GET	/api/health	Health Check
POST	/api/audio/upload	Upload Audio
GET	/api/audio/{id}/status	Processing Status
GET	/api/audio/{id}/result	Analysis Result
POST	/api/audio/{id}/ask	Ask Transcript Questions
GET	/api/live-status	Dashboard Metrics

8. Implementation Challenges & Observations

Challenge 1: CPU-Intensive Transcription

The Faster-Whisper model operates on the CPU by default and can require substantial processing time for large recordings.

Observation:

Long recordings may take nearly the same amount of time as the audio duration itself.

Mitigation:

AssemblyAI integration was provided as an alternative high-speed transcription solution.

Challenge 2: Speaker Diarization Accuracy

Speaker identification accuracy varies depending on the selected diarization method.

Observation:

Fallback heuristic methods cannot match the accuracy of Pyannote or AssemblyAI.

Mitigation:

Priority was given to Pyannote and AssemblyAI whenever configured.

Challenge 3: Background Processing

The MVP uses FastAPI BackgroundTasks for processing uploaded files.

Observation:

Heavy AI workloads execute within the application process and may impact performance under high load.

Mitigation:

Future versions should migrate processing to dedicated worker queues.

Challenge 4: Semantic Search Reliability

Semantic retrieval depends on FAISS and Sentence Transformer availability.

Observation:

If these components fail, retrieval quality decreases significantly.

Mitigation:

A TF-IDF fallback retrieval mechanism was implemented.

9. Results & MVP Outcome

The completed MVP successfully demonstrates a full AI Voice Intelligence workflow.

The system can:

- Process uploaded audio recordings
- Generate speech transcripts
- Identify multiple speakers
- Create conversation summaries
- Extract business insights
- Store analysis results
- Support transcript-based semantic question answering

The platform transforms raw audio recordings into structured business intelligence that can assist sales teams, support teams, and research analysts.

The MVP validates the technical feasibility of automated conversation intelligence using modern AI technologies.

10. Current Limitations & Future Enhancements

Current Limitations

- **Web Process Blocking**

Heavy AI tasks execute inside the FastAPI process.

- **Non-Durable In-Memory Fallback**

Data may be lost if MongoDB becomes unavailable.

- **CPU-Bound Processing**

Local transcription speed is limited without GPU acceleration.

- **Fallback Diarization**

Heuristic speaker labeling has reduced accuracy.

- **Retrieval Fallback**

TF-IDF retrieval lacks true semantic understanding.

Future Enhancements

- Celery worker architecture
- Redis task queue
- GPU-accelerated transcription
- Real-time audio streaming
- Authentication and authorization
- Advanced analytics dashboards
- Sentiment analysis
- CRM integrations
- Multi-language support
- Real-time monitoring

11. Key Learnings

The project provided practical experience in building production-oriented AI systems that combine multiple machine learning components into a single workflow.

Key learnings include:

- Designing modular AI pipelines
- Working with speech recognition systems
- Implementing speaker diarization
- Using LLMs for business intelligence generation
- Building semantic retrieval systems using vector databases
- Integrating cloud AI services with local AI models
- Managing performance challenges in AI applications

The combination of open-source technologies and cloud APIs proved effective for rapid MVP development while maintaining flexibility for future expansion.

12. Deliverables

The following deliverables were completed as part of the project:

Source Code

API Documentation

System Architecture Diagram

Sample Outputs

- Transcript JSON
- Summary Output
- Insight Extraction Output
- Question Answering Output

Supporting Assets

- Sample Audio Recordings
- Environment Configuration Files
- Setup Documentation
- Deployment Instructions

13. Conclusion

The AI Voice Intelligence Platform successfully demonstrates an end-to-end AI-powered conversation analysis workflow. The system combines speech recognition, speaker diarization, large language models, semantic retrieval, and persistent storage to transform raw audio recordings into structured business intelligence.

The completed MVP serves as a strong foundation for future production-scale enhancements and validates the potential of AI-driven voice analytics for sales, customer support, and research applications.

